

新时代文学 LLM Wiki 应用落地方案手册

双轮驱动：面向大众普及的 RAG 导读交互与面向数字人文研究的量化统计

本手册提供了将 “新时代文学 LLM Wiki” 应用于文学大众普及推广与学术批评研究的具体落地方案，包含完整可运行的 Prompt 架构、RAG 交互逻辑，以及面向数字人文（Digital Humanities）研究的 Python 统计脚本。

方案一：大众文学普及 —— 基于 RAG 的“文学导读 AI 助手”

针对大众读者对文学作品低门槛、趣味性与场景化的导读需求，构建基于 RAG 的“导读 AI 助手” Prompt 规范与交互流程。

💡 交互流程设计 (Workflow)

- 提问解析**：接收读者自然语言提问（如“我想知道近年来关于新山乡巨变的长篇小说有哪些？”）。
- 向量与标签检索**：检索 Wiki 库中带有 tags：[主题/新山乡巨变] 及匹配段落。
- 上下文注入与生成**：将匹配的 .md 文本与 YAML 元数据拼接到 Prompt 中，由 LLM 生成带双链引用的导读卡片。

1. 系统 Prompt 与 Python 导读生成函数

```
# 系统提示词 (System Prompt)
SYSTEM_PROMPT = """
你是一个“新时代文学 LLM Wiki”的导读 AI 助手。
你的目标是向普通读者普及新时代文学（2012年至今）优秀作品、作家与文学事件，并为学术研究者提供准确的文献线索。

【交互与回答原则】
1. **基于知识库回答**：只根据检索到的 Markdown Wiki 上下文进行回答。如果知识库中没有明确记载，请诚实说明。
2. **结构化与通俗化结合**：
   - 先用一句话给出核心结论或推荐（适合公众快速浏览）。
   - 提供 2-3 个核心切入点（如作品特色、时代背景、获奖情况）。
   - 附带相关实体链接（如 [[作品名]]、[[作家]]、[[发表期刊]]），引导读者拓展探索。
3. **语言风格**：亲切、专业、富有文学色彩，严禁使用晦涩复杂的学术黑话，但要保持客观准确。
"""

# RAG 检索与回答生成函数
def generate_guided_response(query: str, retrieved_chunks: list[dict]) -> str:
    """
    构建带上下文的 RAG 提示词并调用 LLM 生成回答
    """
    context_str = "\n\n".join([
        f"--- 来源文件: {c['source']} ---\n{c['content']}"
        for c in retrieved_chunks
    ])

    user_prompt = f"""
    【检索到的 wiki 文献上下文】
    {context_str}

    【读者提问】
    {query}

    请按照导读 AI 助手的原则，为读者提供清晰、生动且准确的解答。
    """
    return call_llm_api(system_prompt=SYSTEM_PROMPT, user_prompt=user_prompt)
```

2. 样例交互场景展示

读者输入

LLM 导读助手输出 (结合 Wiki 元数据)

“推荐几部展现南方地域文化的获奖作品？”	📖 核心推荐：[[《繁花》]] 与 [[《燕食记》]] • [[《繁花》]] (金宇澄 著)：获得 [[第九届茅盾文学奖]]。作品以沪语体独特韵味叙述上海市民生活变迁，被誉为新时代沪上沪语文学的代表作。 • [[《燕食记》]] (葛亮 著)：以岭南饮食文化为切入点，生动勾勒出粤港澳大湾区的世纪沧桑历史与风土人情。 🔍 点击实体可继续查看发表阵地 [[《收获》]] 及关联批评文章。
----------------------	---

方案二：学术批评研究 —— 数字人文 Python 量化统计

针对文学研究者，本方案利用 Python 解析 Wiki 库中的 YAML Frontmatter 结构化数据，进行数据清洗、多维统计与可视化呈现，帮助批评家开展“远读（Distant Reading）”与宏观文文学生态分析。

1. 数字人文文献统计 Python 脚本

```
import os
import yaml
import glob
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

def parse_wiki_frontmatter(wiki_dir: str) -> pd.DataFrame:
    """
    遍历 Wiki 目录中的所有 Markdown 文件，提取 YAML Frontmatter 元数据
    """
    records = []
    md_files = glob.glob(os.path.join(wiki_dir, "**/*.md"), recursive=True)

    for file_path in md_files:
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()
            if content.startswith('---'):
                parts = content.split('---', 2)
                if len(parts) >= 3:
                    try:
                        metadata = yaml.safe_load(parts[1])
                        if isinstance(metadata, dict):
                            metadata['file_path'] = file_path
                            records.append(metadata)
                    except yaml.YAMLError as e:
                        print(f"Error parsing {file_path}: {e}")

    return pd.DataFrame(records)

# 1. 加载并清洗元数据
df = parse_wiki_frontmatter("./wiki")

# 2. 定量统计：各主流期刊发表的新时代文学重点作品数量
plt.figure(figsize=(10, 5))
sns.countplot(data=df[df['type'] == 'work'], x='first_venue',
order=df['first_venue'].value_counts().index)
plt.title("新时代文学重点作品发表阵地分布图", fontsize=14)
plt.xlabel("发表期刊/阵地")
plt.ylabel("作品篇数")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("venue_analysis.png", dpi=300)
```

2. 可视化分析产出（示例图表）

下图展示了通过上述 Python 脚本解析 Wiki 目录下作品 YAML 元数据后，自动生成的重点作品发表期刊分布柱状图：

研究价值延伸

- **趋势发现：**快速识别《收获》、《人民文学》、《花城》等核心期刊在新时代作品培育中的权重。
- **交叉检索：**可进一步结合 region（地域）与 genre（流派）进行双因素分析或交叉表统计，为文学批评提供坚实的数据支撑。